Dein nächstes Portfolio-Projekt: SupportOps AI

Repository-Name: `supportops-ai`

Konzept: Ein Support-Copilot für einen fiktiven Onlinehändler. Er analysiert Tickets, durchsucht eine Wissensbasis, fragt Bestelldaten ab und erstellt einen belegten Antwortentwurf. Aktionen wie eine Erstattungsanfrage benötigen eine menschliche Freigabe.

Das baut direkt auf beiden Projekten auf:

Bisherige Erfahrung	Nächste Herausforderung
Ticket-Klassifikation	Antwortentwurf mit Quellen und überprüfbaren Geschäftsregeln
Google-Sheet-Tools	Mehrere typisierte Tools mit getrennten Lese- und Schreibrechten
Redis/RQ	Wiederholbare, idempotente Dokumentverarbeitung
Chatverlauf	Persistierter Workflow mit Freigabe und Wiederaufnahme
20 Evaluationsfälle	Getrennte Retrieval-, Antwort- und Workflow-Evaluation
Docker/CI	Vollständig startbare und öffentlich demonstrierbare Anwendung

Beispielticket:

> „Meine Bestellung AB-0042 ist beschädigt angekommen. Kann ich Ersatz bekommen?“

Das System:

- Klassifiziert das Anliegen.
- Ruft die Bestellung über eine simulierte Shop-API ab.
- Sucht die passende Regel für beschädigte Lieferungen.
- Erstellt einen Antwortentwurf mit Quellen.
- Schlägt gegebenenfalls eine Ersatzlieferung vor.
- Wartet auf die Freigabe eines Supportmitarbeiters.
- Führt die genehmigte Aktion genau einmal aus.

Empfohlener Stack

Komponente	Empfehlung	Zweck
Backend	FastAPI, Pydantic	Bekanntes Fundament weiter vertiefen
Hauptdatenbank	PostgreSQL, SQLAlchemy, Alembic	Tickets, Nutzer, Aktionen und Dokumentmetadaten
Retrieval	Qdrant	Dense- und Sparse-Suche, Filter und Ranking
Workflow	LangGraph	Explizite Zustände, Verzweigungen, Checkpoints und Freigaben
LLM	Zunächst ein Provider hinter einer eigenen Schnittstelle	Qualität und Fehlerverhalten beherrschbar halten
Hintergrundjobs	Redis/RQ	Bereits vorhandene Kenntnisse für Ingestion nutzen
Dokumentverarbeitung	Text-PDFs und Markdown	Einstieg ohne sofortige OCR-Komplexität
Tests und Qualität	pytest, HTTPX, Ruff, mypy oder Pyright	Systemgrenzen und Refactorings absichern
Observability	Strukturierte Logs und OpenTelemetry; optional Langfuse	Schritte, Fehler, Tokenverbrauch und Laufzeit untersuchen
Oberfläche	Kleine React/TypeScript-App	Tickets, Quellen und Freigaben demonstrieren
Betrieb	Docker Compose, GitHub Actions, später eine Cloud	Reproduzierbarkeit und Deployment nachweisen

Die Nachfrage nach Backend, Retrieval, Evaluation und Cloud wird durch die Junior-Anzeige gestützt; LangGraph, Qdrant und FastAPI erscheinen außerdem in den recherchierten erfahreneren Rollen. **Du musst deshalb nicht LangChain, LlamaIndex und LangGraph gleichzeitig lernen.** AI Products (LLM & RAG) (m/f/d)

Für dieses Projekt würde ich LangGraph wählen und Retrieval zunächst direkt mit dem Qdrant-Client implementieren. So lernst du die einzelnen Schritte, bevor du weitere Abstraktionen einführst.

Die Implementierung in sieben Etappen

Etappe 1: Fachliche Regeln und sichere API

Definiere zuerst Kategorien, Eskalationsregeln und erlaubte Aktionen. Erstelle Nutzer, Tickets und eine simulierte Shop-API mit Bestellungen.

Erstes Abschlusskriterium: Nutzer A kann weder Tickets noch Jobs von Nutzer B lesen. Der komplette Grundablauf funktioniert ohne LLM.

Etappe 2: Versionierte Dokument-Ingestion

Importiere beispielsweise 30–50 selbst erstellte Markdown-Dateien und Text-PDFs: Rückgabe, Lieferung, beschädigte Ware, Rechnungen und technische FAQ.

Speichere Dokument-ID, Version, Hash, Abschnitt beziehungsweise Seite und Zugriffskontext. Danach folgen Chunking, Embeddings und Indexierung.

Abschlusskriterien:

- Derselbe Import erzeugt keine Duplikate.
- Eine neue Version ersetzt veraltete Suchinhalte.
- Fehlerstatus und erneuter Versuch sind nachvollziehbar.
- Jeder Chunk lässt sich zur Originalquelle zurückverfolgen.

Etappe 3: Retrieval vor Antwortgenerierung

Baue zunächst einen Suchendpunkt, der relevante Abschnitte zurückgibt.

Vergleiche anschließend:

- Dense Search für semantische Ähnlichkeit.
- Sparse/BM25-Suche für exakte Begriffe und Produktkennungen.
- Hybrid Search mit Reciprocal Rank Fusion.
- Optional einen Reranker.

Qdrant unterstützt die Kombination mehrerer Suchläufe und Fusion direkt über seine Query API. Ob Hybrid Search bei deinem Datensatz besser ist, musst du messen. Qdrant

Wichtig: Zugriffsfilter gelten bereits während der Suche. Fremde Inhalte dürfen nicht erst aus dem fertigen LLM-Kontext entfernt werden.

Etappe 4: Belegte Antwortentwürfe

Das LLM erhält Ticket, Bestelldaten und Suchergebnisse. Es liefert ein validiertes Ergebnis mit Antwortentwurf, Quellen-IDs und gegebenenfalls einem Aktionsvorschlag.

Prüfe im Code, dass angegebene Quellen tatsächlich abgerufen wurden. Prüfe zusätzlich anhand von Testfällen, ob sie die Aussagen inhaltlich stützen – eine gültige Quellen-ID allein genügt nicht.

Abschlusskriterium: Fehlen ausreichende Informationen, fragt das System nach oder eskaliert.

Etappe 5: Tools und LangGraph

Implementiere wenige klar begrenzte Tools:

- `get_order`
- `get_shipment_status`
- `search_knowledge`
- `propose_replacement`
- `create_escalation`

Der Workflow enthält Klassifikation, Recherche, Entwurf, Prüfung und gegebenenfalls Freigabe. Nutze persistente Checkpoints, damit ein pausierter Lauf nach einem Neustart fortgesetzt werden kann. LangGraph unterstützt dieses Muster über Interrupts und seine Persistenzschicht. Docs by LangChain

Abschlusskriterien:

- Schreibaktionen erfolgen erst nach berechtigter Freigabe.
- Freigegeben wird ein konkreter, gespeicherter Aktionsvorschlag.
- Wiederaufnahme und Wiederholung erzeugen keine doppelte Aktion.
- Tool-Argumente und Rechte werden im ausführenden Code geprüft.

Etappe 6: Evaluation und Fehlerexperimente

Erstelle etwa 100–150 sorgfältig geprüfte Fälle. Trenne Entwicklungsfälle von einem zurückgehaltenen Testset.

Ebene	Was du misst
Klassifikation	F1 pro Kategorie, Recall für dringende Fälle
Retrieval	Recall@5 und MRR auf markierten relevanten Abschnitten
Antworten	Korrektheit, Quellenunterstützung, Verhalten bei fehlenden Informationen
Workflow	Richtige Tools, richtige Argumente, Freigabe und Eskalation
Betrieb	Fehlerrate, p50/p95-Latenz, Tokenverbrauch und Kosten

Teste bewusst Timeouts, Rate Limits, ungültige Ausgaben, manipulierte Dokumentinhalte, fremde Ressourcen und einen Neustart während der Freigabe.

Ein LLM-Judge kann qualitative Bewertungen ergänzen. Er ersetzt weder deterministische Checks noch manuell geprüfte Beispiele.

Etappe 7: Deployment und Portfolio-Aufbereitung

Die Anwendung sollte mit einem dokumentierten Befehl startbar sein. Danach folgt ein Cloud-Deployment mit Demo-Daten und begrenzter Nutzung.

Für Bewerbungen gehören dazu:

- README mit Architektur, Startanleitung und Grenzen.
- Kurzes Demo-Video.
- Dokumentierte Evaluationsdaten und Ergebnisse.
- Vergleich von Dense, Sparse und Hybrid Search.
- Zwei bis drei erklärte technische Entscheidungen.
- Ein Beispiel für einen gefundenen Fehler und seinen Regressionstest.

Plane dafür grob **12–16 Wochen mit etwa 15–20 konzentrierten Stunden pro Woche**; bei weniger Zeit entsprechend länger. Jede Etappe sollte einen demonstrierbaren Zwischenstand liefern.